

ACTIVITY 5 - ARRAYS AND OBJECTS

branch name: js-activity-5

ACTIVITY 5 PART 1 - ARRAYS AND OBJECTS

Instruction

Task 1

Declare a variable called **addressBook** and set it as an empty **object**. This will be your address book container.

```
_____ addressBook = {};
```

Task 2

Add a contact information to your address book **object**

```
addressBook.name = _____;
```

```
addressBook.phoneNumber = _____;
```

Task 3

Retrieve Contact Information

Use **console.log()** to retrieve and print the name and phone number of the contact you just added

```
console.log('Name: ', addressBook.name);
```

ACTIVITY 5 PART 1 - ARRAYS AND OBJECTS

Instruction

Task 4

Update Contact Information

Change the phone number of a contact property in your address book to a new number

Task 5

Retrieve and Display Updated Contact Information

Using `console.log()`, print out the updated contact's name and phone number

Extra Challenge

Create a function that can change phone number and name

ACTIVITY 5 PART 2 - ARRAYS AND OBJECTS

Task 1: Create variable

Declare a variable called **bankAccount** and set it as an empty **object**. This will be your bank account

```
___ bankAccount = ___;
```

Task 2: Add property for balance

Add a property called **balance** to your bank account and set an initial balance value (e.g. 1000);

Task 3: Create a deposit method

Add a method called **deposit** to the **bankAccount** **object**. This method should take an amount of money as a **parameter** and add it to the current balance.

```
// balance = 100
```

```
// Invoke the deposit method
```

```
bankAccount.deposit(200);
```

```
// Check our balance
```

```
console.log(bankAccount.balance); // 👉 300
```

ACTIVITY 5 PART 2 - ARRAYS AND OBJECTS

Task 4: Create a withdraw method

Add a method called **withdraw** to the **bankAccount** object. This method should take an amount of money as a **parameter** and subtract it from the current balance.

You cannot withdraw an amount that is less than the current balance.

```
// balance = 300

// Invoke the withdraw method

bankAccount.withdraw(100);

// Check our current balance

console.log(bankAccount.balance); // 👉 200

// Ensure you have sufficient funds to withdraw

// Otherwise nothing will happen

bankAccount.withdraw(300);

// Check our current balance

console.log(bankAccount.balance); // 👉 200
```

Extra Challenge

- Implement error handling in the **withdraw** method to prevent overdrawing (withdrawal of more money than the balance)
- Implement a transaction fee per withdraw, let say every invoke of **withdraw** method, it will charge 5 that will be deducted to the

ACTIVITY 5 PART 3 - ARRAYS AND OBJECTS

Task 1: Create an empty array

Declare a variable called `favoriteFruits` and set it as an empty `array`. This will be your `array` for favorite fruits.

```
_____ favoriteFruits = _____;
```

Task 2: Add initial elements

Add the elements apple, banana and orange to the `favoriteFruits` `array`. You can add different fruits.

Task 3: Add more elements

Use the `push()` method to add more favorite fruits to the `array`

Task 4: Log elements

Use a loop to log each element in the `favoriteFruits` `array` to the console

ACTIVITY 5 PART 4 - ARRAYS AND OBJECTS

Instruction

A vending machine can be represented as an **object** that has snacks and a function to vend.

The task is to create a **VendingMachine** class with the following properties:

- **snacks** is an **array** of snacks. And a snack is an **object** with a name and a price.
- **vend** is a function that allows a customer to enter (1) the **array** position of the snack the customer wants and (2) the bill that the customer will pay. This function will then return an object representing the snack's name and the customer's change.

And then, initialize the **VendingMachine** to have 3 snacks.

ACTIVITY 5 PART 4 - ARRAYS AND OBJECTS

Test Cases

Example that vendingMachine.snacks was initialized to have

```
[  
  { name: "Cheese Ring", price: "15" },  
  { name: "Nova", price: "12" },  
  { name: "Moby", price: "5" },  
];  
vendingMachine.vend(1, 20); // should return {name: "Nova", change: 8}  
vendingMachine.vend(0, 20); // should return {name: "Cheese Ring", change: 5}  
vendingMachine.vend(5, 20); // should return "Snack not found"  
vendingMachine.vend(2, 2); // should return "Your money is not enough, sorry"
```


ACTIVITY 5 PART 4 - ARRAYS AND OBJECTS

Test Cases

Example that vendingMachine.snacks was initialized to have

```
[  
  { name: "Cheese Ring", price: "15" },  
  { name: "Nova", price: "12" },  
  { name: "Moby", price: "5" },  
];  
vendingMachine.vend(1, 20); // should return {name: "Nova", change: 8}  
vendingMachine.vend(0, 20); // should return {name: "Cheese Ring", change: 5}  
vendingMachine.vend(5, 20); // should return "Snack not found"  
vendingMachine.vend(2, 2); // should return "Your money is not enough, sorry"
```